

Topic 6.4

Constructors are allowed to have an initializer clause. For example:

```
class A
{
    public:
        A();           // no-argument constructor
        A( int x );    // another constructor
        ...
};

class B : public A
{
    public:
        B();           // no-argument constructor
        B( int x, float y ); // another constructor

    private:
        float internal_y;
};

B::B( int x, float y ) : A(x), internal_y(y)
{
    ...
};
```

In the definition of `B::B`, the code following the colon `:` before the body of the function is an *initializer list*. There are two fundamental things that can be done in an initializer list:

- You can specify a particular constructor in the base class to be invoked. If no particular constructor is called, the no-argument constructor in the base class is invoked.
- You can initialize a data member to a particular value.

Technically, the constructor for the derived class is invoked first. However, prior to executing the body of that constructor, the constructor for the base class is invoked, and has effect, before the body of the derived class's constructor is executed.

In a multi-level derivation, the constructors are invoked from the derived class toward the highest-level base class, but the effect is that the body of the highest-level base class constructor is executed first, then the body of the next highest level base class constructor, etc., with the body of the most-derived class executing last.

```
// MODULE      inherit1.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    inherit1.cpp
// PURPOSE:
```

```
// Illustrate inheritance.
//

#include <iostream>
#include <stdlib.h>
#include <string.h>

class Thing    // Thing is the base of all our objects.
{
    public:
    Thing() { strcpy(name,""); };
    Thing( char * thing_name )
    { strcpy(name,thing_name); };
    void print();

    protected:
    char name[80];
};

void Thing::print()
{
    cout << "name:" << name;
}

class Person : public Thing
{
    public:
    Person();
    Person( char *person_name, int person_age );
    void print();

    protected:
    int age;
};

Person::Person() : Thing()
{
    age = -1;
}

Person::Person
( char *person_name, int person_age ) :
    Thing(person_name)
{
    age = person_age;
}

void Person::print()
{
    Thing::print();
    cout << " age: " << age;
}

class Politician : public Person
{
    public:
    Politician( char *person_name, int person_age, char *pol_party );
    void print();

    protected:
    char party[40];
};

Politician::Politician( char *person_name,
    int person_age,
    char *pol_party )
    : Person(person_name,person_age)
```

```

    {
        strcpy(party,pol_party);
    }

void Politician::print()
{
    Person::print();
    cout << " party:" << party;
}

int main(void)
{
    Thing      t("Cessna 182");
    Person      me("Gary J. Blair",37);
    Politician  president("Bill Clinton",
46,"Democrat");
    Thing      *p; // Can point at Thing or
// anything derived from Thing.

    t.print();      cout << "\n";
    me.print();      cout << "\n";
    president.print(); cout << "\n";

    p = &t;
    p->print();
    cout << endl;

    p = &me;
    p->print();
    cout << endl;

    p = &president;
    p->print();
    cout << endl;

    return 0;
}

```

Output from inherit1.cpp:

```

name:Cessna 182
name:Gary J. Blair age: 37
name:Bill Clinton age: 46 party:Democrat
name:Cessna 182
name:Gary J. Blair
name:Bill Clinton

```

Question for students: why are the last 3 lines of output in the preceding program "incomplete?" The answer will be discussed shortly.

Topic 6.5 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-6-dot-5>)